

Research Methodology in CS (CSEG3060_3)

Lecture 3: Essential Attributes and Errors in Selection of a Research Problem

Unit 1: Introduction to Research Problem • Mapped Course Outcome: CO1

Lecture 3: Essential Attributes and Errors in Selection of a Research Problem

Course: Research Methodology in Computer Science (CSEG3060_3) **Program:** B.Tech (Computer Science) **Semester:** V **Unit:** I — Introduction to Research Problem **Mapped Course Outcome:** CO1

1. Lecture Overview and Learning Objectives

By the end of this lecture, the learner should be able to:

- Describe the essential attributes that distinguish a scientifically sound research problem from a trivial or unworkable one.
 - Apply a structured set of evaluative criteria when shortlisting a research problem in computer science.
 - Identify and diagnose the common errors researchers commit at the problem-selection stage.
 - Reframe a poorly stated problem into a well-defined one using practical heuristics.
 - Connect the abstract qualities of a "good" problem to concrete CS research scenarios (algorithms, networks, AI/ML, software engineering).
-

2. Introduction

The selection of a research problem is the single most consequential decision in the research lifecycle. An elegant methodology, a sophisticated toolchain, or a powerful statistical model cannot rescue a study built on a poorly chosen problem. Conversely, a well-selected problem—even when investigated with modest resources—tends to produce useful, publishable, and citable contributions.

This lecture operationalises the principle by examining two complementary questions:

1. *What makes a research problem good?* — The essential attributes.

2. *What makes a research problem bad?* — The selection errors that derail studies before they begin.

The discussion is anchored throughout in examples drawn from computer science so that abstract methodological language is consistently mapped to recognisable engineering and scientific situations.

3. Section I — Essential Attributes of a Good Research Problem

A "good" research problem in computer science satisfies a constellation of attributes. These attributes are interrelated; weakness in one often compromises the others. The attributes can be grouped under three meta-categories: **Conceptual Quality**, **Operational Feasibility**, and **Strategic Value**.

3.1 Conceptual Quality

3.1.1 Clarity and Precision of Statement

A research problem must be expressed in unambiguous, operationally defined terms. Vague verbs such as "study," "understand," "look at," or "explore the area of" should be replaced with precise verbs such as "evaluate," "compare," "predict," "classify," or "optimise."

Poor: "Study deep learning." *Improved:* "Evaluate the impact of activation-function selection (ReLU, GELU, Swish) on the convergence speed of residual CNNs trained on CIFAR-10."

3.1.2 Novelty and Originality

The problem should advance the frontier of knowledge. Novelty, however, is relative and can be expressed along several dimensions:

- **Theoretical novelty** — proposing a new model, proof, or complexity bound.
- **Methodological novelty** — applying an existing method to a new domain, or refining a method in a non-trivial way.
- **Empirical novelty** — generating new evidence on a contested question, e.g., benchmarking a new optimiser across heterogeneous hardware.
- **Application novelty** — deploying an established technique to solve a previously unsolved practical problem.

A problem need not be novel in every dimension; CS theses routinely make methodological or empirical contributions.

3.1.3 Theoretical Grounding

A research problem should arise from an identifiable body of literature and contribute back to it. The researcher must be able to position the work with respect to at least one established theory (e.g., PAC learning, information theory, queueing theory, type theory, software metrics theory).

3.1.4 Testability and Falsifiability

Following Popperian epistemology, a problem should admit hypotheses that can, in principle, be refuted. In CS this translates to questions whose answers can be supported by measurable outcomes:

- Does the proposed algorithm reduce worst-case time complexity?
- Does the proposed model achieve statistically significant gains in F1-score on a held-out test set?
- Does the proposed protocol lower end-to-end latency under specified load conditions?

A problem that cannot, even in principle, be refuted is closer to speculation than to research.

3.2 Operational Feasibility

3.2.1 Researchability

The problem must be amenable to investigation using available or obtainable methods. A problem that demands access to a hyperscale data centre or to proprietary industrial control systems may be theoretically interesting but practically unresearchable for a student.

3.2.2 Feasibility of Data Acquisition

Empirical CS research depends on data—whether benchmark datasets, log files, survey responses, or simulation traces. The problem must specify the data sources and confirm that they are accessible, ethical, and of adequate volume and quality.

For quantitative work, a rough sample-size awareness is useful. For comparing two means with expected effect size d , standard normal approximation suggests a per-group sample size of:

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2}{d^2}$$

$$n \approx d^{-2} 2(z_{1-\alpha/2} + z_{1-\beta})^2$$

where $z_{1-\alpha/2}$ and $z_{1-\beta}$ are the standard normal deviates corresponding to the significance level and statistical power, respectively. A problem whose data requirements vastly

exceed n attainable is not feasible.

3.2.3 Compatibility with Researcher's Competence

The problem should lie within (or just beyond) the researcher's existing skill envelope. A novice attempting quantum algorithm design or category-theoretic semantics without adequate preparation is unlikely to produce a credible study.

3.2.4 Manageability of Scope

A common failure mode is over-scoping. The scope must be commensurate with the time, computing resources, and human resources available. A manageable problem is one that can produce at least one defensible contribution within the stipulated duration.

3.2.5 Ethical Acceptability

The problem must conform to institutional ethical norms. In CS this includes responsible data handling (privacy, consent), avoidance of harm from deployed AI systems, and compliance with IRB or equivalent ethics committee requirements.

3.3 Strategic Value

3.3.1 Relevance and Significance

The problem should matter—to the discipline, to industry, or to society. Significance can be established by demonstrating:

- A gap in the literature explicitly acknowledged by prior surveys.
- A practical pain point documented in industry reports or case studies.
- A theoretical contradiction or anomaly that warrants resolution.

3.3.2 Timeliness

The problem should address a contemporary concern. While foundational work is always valuable, applied CS problems are particularly time-sensitive because of rapidly evolving platforms (e.g., LLM architectures, edge devices, 6G networks).

3.3.3 Generative Potential

A good problem generates further questions. A truly fertile problem, once solved, opens three new avenues rather than closing the field.

3.3.4 Interest and Motivation

The researcher's intrinsic interest is not a luxury but a strategic asset: it sustains the inevitable periods of dead-end debugging, negative results, and reviewer criticism.

4. Section II — A Synthesis: The "ATTR-FEAS" Checklist

The attributes above can be consolidated into a mnemonic heuristic for use during problem selection and proposal review.

Letter	Attribute	Diagnostic Question
A	Aim clarity	Can the problem be stated in one unambiguous sentence?
T	Testability	Can at least one falsifiable hypothesis be formulated?
T	Theoretical link	Is the problem grounded in an identifiable literature?
R	Relevance	Does the problem matter to a recognised community?
F	Feasibility	Are data, tools, and skills realistically available?
E	Ethical soundness	Does the study pass an ethics review?
A	Achievability of scope	Can the work be completed in the allotted time?
S	Significance of contribution	Will the outcome advance knowledge or practice?

A problem scoring affirmatively on at least seven of the eight items is generally defensible.

5. Section III — Errors in the Selection of a Research Problem

Errors of selection are categorised into five families, each illustrated with a typical CS scenario.

5.1 Errors of Scope

5.1.1 The Over-Broad Problem

Symptom: The problem statement covers an entire subfield. *Example:* "Improve the performance of deep neural networks." This cannot be answered in any finite study. *Remedy:* Narrow by algorithm, dataset, hardware, or task.

5.1.2 The Over-Narrow Problem

Symptom: The problem addresses a trivial special case. *Example:* "Optimise a bubble sort routine on a 32-element array of integers." The result is unlikely to generalise or interest anyone. *Remedy:* Generalise by parameter, structure, or domain until the result becomes indicative of a class of phenomena.

5.2 Errors of Conceptual Grounding

5.2.1 Insufficient Literature Review

Selecting a problem without an adequate literature scan frequently leads to duplication. A problem that has already been solved—even if the solver is unfamiliar with the work—is a wasted effort. *Remedy:* Conduct a systematic mapping study or scoping review prior to finalising the problem.

5.2.2 Absence of a Theoretical Framework

A problem unanchored in theory produces descriptive but not explanatory results. *Example:* Reporting that Model A outperforms Model B without invoking any learning-theoretic explanation. *Remedy:* Identify at least one theoretical lens (e.g., bias-variance decomposition, VC dimension, software evolution theory) and design the study around it.

5.3 Errors of Empirical Design

5.3.1 Unavailability of Data

A problem that requires proprietary, sensitive, or practically inaccessible data is non-researchable. *Example:* A thesis requiring millions of patient ECG records without a hospital collaboration. *Remedy:* Either secure the data through formal agreements or reframe the problem around publicly available surrogates.

5.3.2 Inadequate Measurement Instruments

Selecting a problem whose constructs cannot be reliably measured yields noisy, uninterpretable results. *Example:* Attempting to measure "code quality" using only lines of code. *Remedy:* Validate the measurement instruments (e.g., through inter-rater reliability for human-coded data, or established benchmarks for algorithmic comparison).

5.4 Errors of Motivation and Context

5.4.1 Fad-Driven Selection

Selecting a problem because it is fashionable—e.g., "I want to work on blockchain" or "I want to fine-tune an LLM"—without a genuine research question leads to shallow work. *Remedy*: Begin with the question, not the technology. The tool should serve the question, not vice versa.

5.4.2 Adviser-Driven Selection without Ownership

A problem imposed by an adviser or funding agency without the student's intellectual buy-in tends to produce low-quality dissertations. *Remedy*: Negotiate ownership of a sub-problem that genuinely interests the researcher while still aligning with the larger grant.

5.4.3 Politically Convenient Topics

Sometimes problems are selected because they are politically safe or aligned with the institution's branding. This can yield work that is technically correct but lacks intellectual edge. *Remedy*: Look for the genuine intellectual tension inside the "safe" topic.

5.5 Errors of Self-Assessment

5.5.1 Problem Beyond Competence

A student attempting to design a novel cryptographic primitive without a background in number theory or complexity theory is unlikely to produce credible work. *Remedy*: Either upskill systematically, or select a problem one tier below the current frontier of the field.

5.5.2 Problem Beneath Interest

A problem selected purely for its ease ("the safe PhD") rarely produces the deep engagement required for original work. *Remedy*: Aim for the intersection of competence, interest, and significance—the classical "Pick Three" trade-off, but with the additional constraint of feasibility.

5.5.3 Ignored Cost and Time Constraints

Selecting a problem whose solution requires a 6-month cluster run on a personal laptop is an error of planning rather than of conception. *Remedy*: Estimate resource budgets (compute hours, storage, personnel, calendar time) before finalising the problem.

5.6 Diagnostic Summary of Errors

Error Family	Typical Symptom	Most Effective Remedy
Scope	Problem too broad or too narrow	Restate with explicit boundary conditions
Conceptual	No theory, no literature anchor	Conduct a systematic literature review
Empirical	No data, no instruments	Pilot data collection early
Motivation	Fad-driven, externally imposed	Reframe around genuine intellectual tension
Self-assessment	Beyond competence, beneath interest	Calibrate scope to researcher profile

6. Section IV — Case Studies in CS

6.1 Case Study A: A Well-Formed Problem in Network Security

Problem: "Evaluate the detection latency and false-positive rate of a transformer-based intrusion detection system against three baselines (signature-based Snort, random forest, and LSTM) on the CICIDS-2017 dataset, under class-imbalanced conditions."

Why this is a good problem:

- **Clear aim** — comparison is operationalised through two metrics.
- **Novelty** — applying transformers to a class-imbalanced IDS task is non-trivial.
- **Feasibility** — CICIDS-2017 is publicly available; the baselines are open-source.
- **Testability** — hypotheses are explicit (e.g., H_0 : no significant difference in detection latency at $\alpha = 0.05$).
- **Relevance** — intrusion detection is a perennial CS security concern.

6.2 Case Study B: A Poorly Formed Problem

Problem: "Investigate artificial intelligence."

Why this fails:

- It violates nearly every attribute: no scope, no testable hypothesis, no theoretical anchor, no feasibility boundary, no measurement.

- It is in fact not a problem but a topic.

Reframed Version: "Quantify the effect of curriculum learning on the sample efficiency of vision transformers fine-tuned for medical image classification under a fixed compute budget of 100 GPU-hours."

6.3 Case Study C: A Problem with Hidden Feasibility Trap

Problem: "Train a foundation model for Indian legal text from scratch and benchmark it against GPT-4."

Why this is problematic:

- The compute cost of pretraining a foundation model is incompatible with a typical B.Tech or MTech thesis budget.
- Benchmarking against GPT-4 introduces intellectual-property and reproducibility issues.

Reframed Version: "Fine-tune an open-source 7B-parameter LLM (e.g., LLaMA-2) on a curated Indian legal corpus and evaluate its performance on three downstream tasks: case summarisation, statutory QA, and judgement classification."

The reframed version retains the original motivation while rendering the study feasible.

7. Summary

The selection of a research problem is not an act of inspiration but an act of disciplined evaluation. A good research problem in computer science is simultaneously **clear, novel, theoretical, testable, feasible, ethical, manageable, and significant**. The ATTR-FEAS checklist operationalises these qualities into a reusable diagnostic. Equally important is the diagnosis of selection errors, which fall into five families: errors of scope, of conceptual grounding, of empirical design, of motivation, and of self-assessment. Recognising these errors early converts a doomed study into a salvageable one through reframing.

A useful closing heuristic: **the best research problem is one that, when stated aloud, a peer in the same subfield can criticise constructively**—because what can be criticised is what can be improved, and what can be improved is what can be defended.

8. Exam Preparation Tips and Practice Questions

8.1 Preparation Tips

- Memorise the ATTR-FEAS heuristic and be able to apply it to any given problem statement.
- Prepare at least two contrasting examples—one well-formed and one poorly formed—from your own area of CS interest (e.g., NLP, computer vision, distributed systems).
- Be able to *reframe* a vague problem into a precise, testable one; examiners often test this skill directly.
- Distinguish between *conceptual* and *operational* errors in problem selection.
- Practise writing hypothesis statements in the null and alternative form, e.g., H_0 versus H_1 , with the corresponding test statistic.

8.2 Short-Answer Questions

1. State and justify any five essential attributes of a good research problem.
2. Differentiate between an over-broad and an over-narrow research problem, giving one CS example of each.
3. Explain why "novelty" should not be confused with "complexity" in problem selection.
4. Why is theoretical grounding important even for an empirically oriented CS thesis?

8.3 Long-Answer / Applied Questions

1. A peer proposes the problem: "Make cloud computing faster." Diagnose the errors in this statement and rewrite it as a well-formed research problem in the field of distributed systems or cloud resource management.
2. For each attribute in the ATTR-FEAS checklist, give one diagnostic question you would pose to a student presenting a research proposal.
3. A funding agency offers a grant to study "AI for healthcare" but specifies that the work must be completed in 18 months with a single GPU workstation. Discuss the ethical, feasibility, and scope-related concerns, and propose a defensible problem formulation.

8.4 Higher-Order Thinking Question

Consider the following research problems. For each, classify the *primary* error (scope, conceptual, empirical, motivational, self-assessment) and propose a one-paragraph reframing that preserves the underlying motivation:

- "Build a chatbot that talks like a human."

- "Prove $P \neq \text{I} = \text{NP}$."
 - "Survey all machine-learning algorithms ever published."
 - "Reimplement TensorFlow from scratch in a new language."
-

9. Suggested Further Reading

- Kothari, C. R., & Garg, G. (2019). *Research Methodology: Methods and Techniques* (4th ed.). New Age International. — Chapters on the research problem.
 - Creswell, J. W., & Creswell, J. D. (2018). *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches* (5th ed.). SAGE.
 - Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer. — Chapter on defining the experimental goal.
 - Kitchenham, B. A., & Charters, S. (2007). *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. EBSE Technical Report.
 - Popper, K. (2002). *The Logic of Scientific Discovery* (reprint). Routledge. — For the falsifiability criterion.
-

End of Lecture 3 Notes.